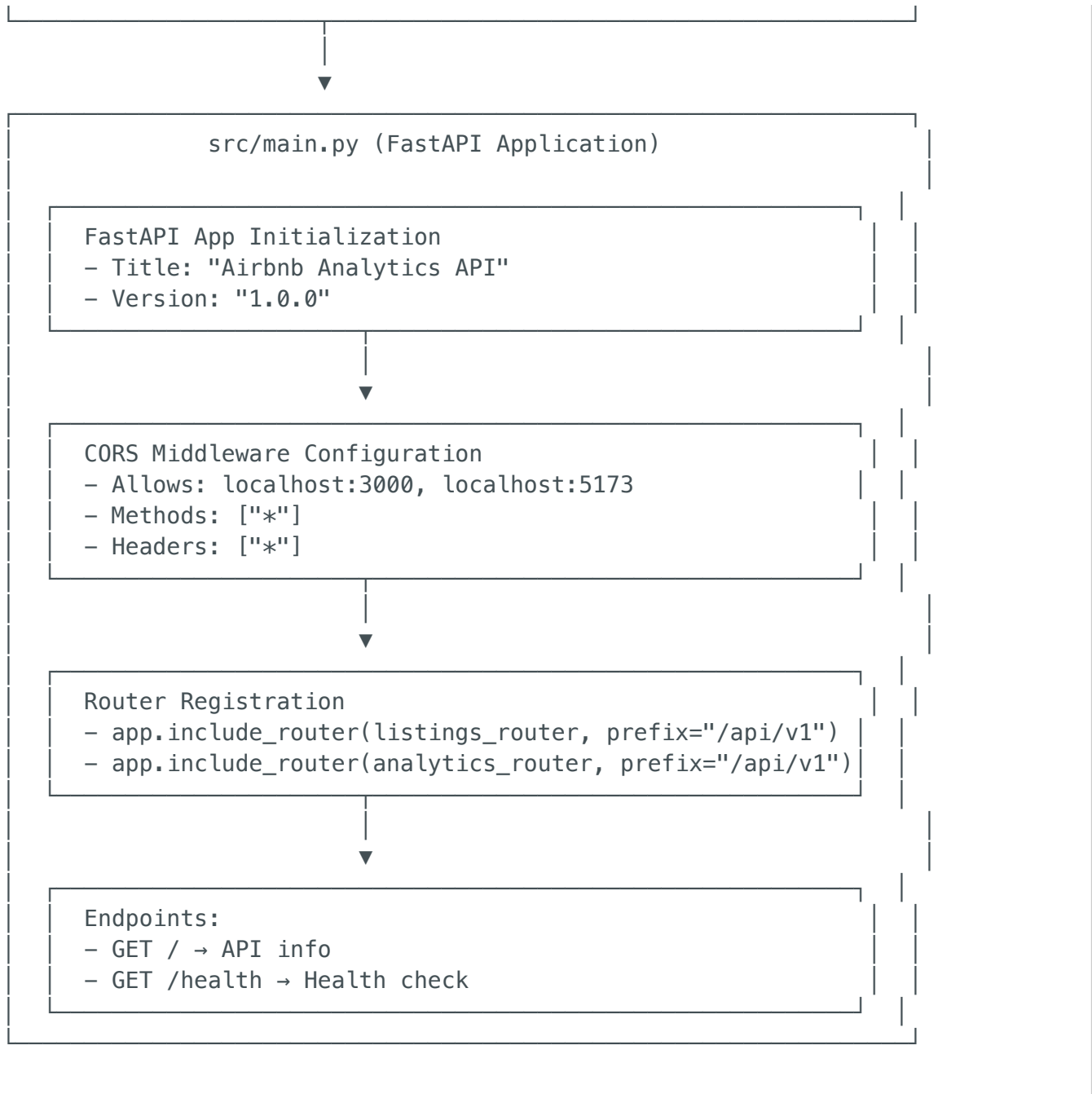
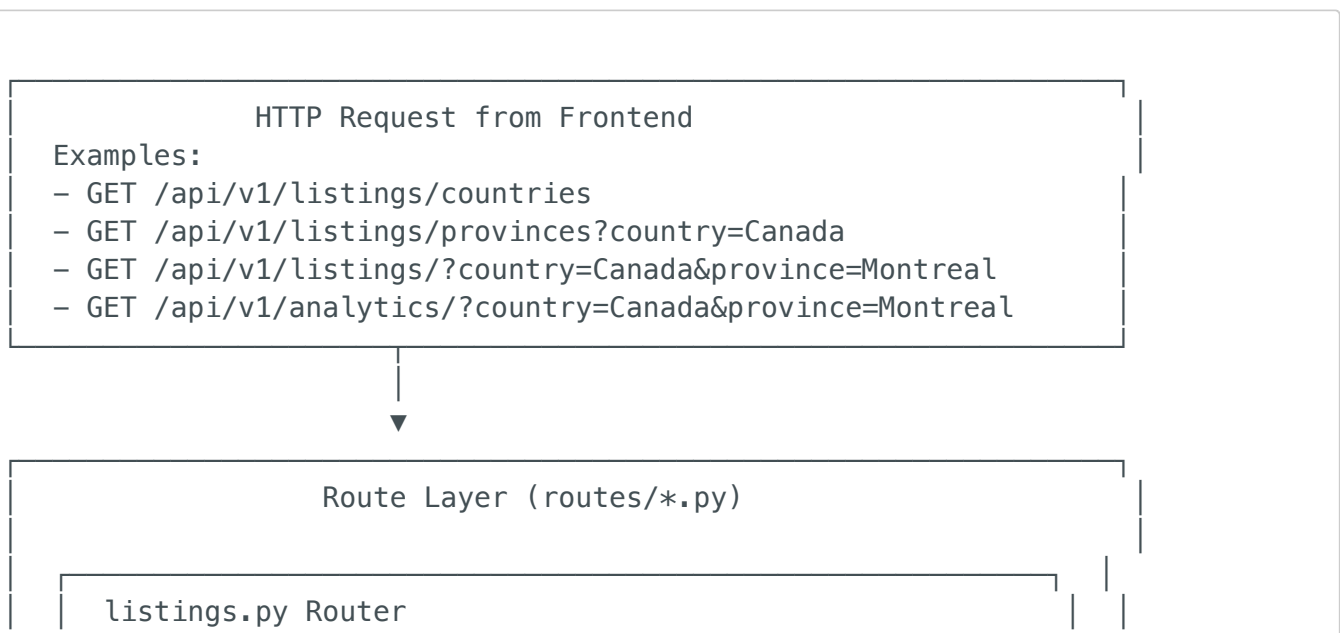

```
run.py (Entry Point)
```

- Starts Uvicorn ASGI server
- Runs on host="0.0.0.0", port=8000, reload=True
- Loads FastAPI app from src.main



API Request Flow



Routes:

- GET /listings/countries → get_countries()
- GET /listings/provinces?country=X → get_provinces()
- GET /listings/?country=X&province=Y → list_listings()

analytics.py Router

Routes:

- GET /analytics/?country=X&province=Y → get_listings_analytics()

Controller Layer (controllers/*.py)

listings_controller.py

Functions:

- get_listings(country, province, dataset)
 - ↳ load_csv()
 - ↳ clean_listings()
 - ↳ _dataframe_to_listings() → ListingResponse

analytics_controller.py

Functions:

- get_analytics(country, province, dataset)
 - ↳ load_csv() (single or multiple provinces)
 - ↳ clean_listings()
 - ↳ analyze_property_type_distribution()
 - ↳ analyze_room_type_distribution()
 - ↳ analyze_amenity_distribution()
 - ↳ analyze_price_statistics()
 - ↳ analyze_review_statistics()
 - ↳ analyze_instant_booking_stats()

Returns: AnalyticsResponse

Data Access Layer (data_access/*.py)

registry.py

Functions:

- list_countries() → ["Canada", "America", ...]
- list_provinces(country) → ["Montreal", "Toronto", ...]
- get_dataset_path(country, province, dataset) → Path

Features:

- Scans Data/ directory structure

- Case-insensitive lookups
- Cached country/province mappings
- Environment variable: DATA_ROOT

loader.py

Functions:

- load_csv(country, province, dataset) → DataFrame

Features:

- Uses registry.get_dataset_path()
- LRU cache (maxsize=32) for CSV files
- Returns pandas DataFrame
- refresh_cache parameter for cache invalidation



Data Cleaning Layer (data_cleaning/*.py)

cleaning_pipeline.py

Functions:

- clean_listings(dataframe, country, province, dataset_name, rules) → DataFrame

Cleaning Steps:

1. Normalize column names to snake_case
2. Apply rename_map (e.g., id → listing_id)
3. Drop duplicates and fully null rows
4. Fill default values (e.g., price: 0.0)
5. Convert numeric columns (price, latitude, etc.)
6. Convert datetime columns (host_since)
7. Convert boolean columns (host_identity_verified)
8. Validate required columns
9. Add timestamps (created_at, updated_at)
10. Add metadata (country, province)

Default Rules:

- Dataset-specific rules in DEFAULT_RULES dict
- Custom rules can be passed via rules parameter



Service Layer (services/*.py)

analytics_service.py

Functions:

- analyze_property_type_distribution(df) → List[Dict]
- analyze_room_type_distribution(df) → List[Dict]
- analyze_amenity_distribution(df, top_n=20) → List[Dict]
- analyze_price_statistics(df) → Dict

- analyze_review_statistics(df) → Dict
- analyze_instant_booking_stats(df) → Dict

Analytics Computations:

- Value counts for categorical fields
- Statistical aggregations (mean, median, min, max)
- Parsing amenities from string/list formats



Schema Layer (schemas/*.py)

listing_schemas.py (Pydantic Models)

Request/Response Models:

- Listing → Individual listing record
- ListingResponse → Envelope with listings array
- CountryListResponse → List of countries
- ProvinceListResponse → List of provinces
- AnalyticsResponse → Complete analytics data

Analytics Models:

- PropertyTypeDistribution
- RoomTypeDistribution
- AmenityDistribution
- PriceStatistics
- ReviewStatistics
- InstantBookingStats



JSON Response to Frontend

- Validated by Pydantic schemas
- FastAPI auto-generates OpenAPI docs



Detailed Request Flow Examples

Example 1: Get Countries List

GET /api/v1/listings/countries



Route: listings.py → get_countries()

Registry: `list_countries()`
- Scans Data/ directory
- Returns folder names: `["America", "Australia", "Canada"]`

Schema: `CountryListResponse(countries=[...])`

JSON Response: `{"countries": ["America", "Australia", ...]}`

Example 2: Get Analytics (Single Province)

GET `/api/v1/analytics/?country=Canada&province=Montreal`

Route: `analytics.py` → `get_listings_analytics()`

Controller: `analytics_controller.py` → `get_analytics()`

1. Load CSV:
 - ↳ `registry.get_dataset_path("Canada", "Montreal", "listings")`
 - ↳ `loader.load_csv()` → DataFrame (cached)
2. Clean Data:
 - ↳ `cleaning_pipeline.clean_listings()` → DataFrame
3. Compute Analytics:
 - ↳ `analyze_property_type_distribution()`
 - ↳ `analyze_room_type_distribution()`
 - ↳ `analyze_amenity_distribution(top_n=20)`
 - ↳ `analyze_price_statistics()`
 - ↳ `analyze_review_statistics()`
 - ↳ `analyze_instant_booking_stats()`
4. Build Response:
 - ↳ `AnalyticsResponse(...)`

JSON Response with all analytics data

Example 3: Get Analytics (All Provinces in Country)

GET /api/v1/analytics/?country=Canada



Controller: analytics_controller.py → get_analytics()

1. List All Provinces:
 - ↳ registry.list_provinces("Canada")
 - ["Montreal", "Toronto", "Vancouver", ...]
2. Load & Clean Each Province:
For each province:
 - ↳ loader.load_csv(country, province, "listings")
 - ↳ clean_listings(...)
3. Combine DataFrames:
 - ↳ pd.concat([df1, df2, ...], ignore_index=True)
4. Compute Analytics on Combined Data:
 - ↳ (same analytics functions as single province)
5. Return Aggregated AnalyticsResponse



Architecture Layers

API Layer (FastAPI)

- main.py: FastAPI app, CORS, router registration
- routes/: HTTP endpoint definitions
- schemas/: Pydantic request/response models



Controller Layer

- Orchestrates business logic
- Coordinates data access, cleaning, and services

- Handles errors and raises custom exceptions

Service Layer

- analytics_service.py: Pure data analysis functions
- Stateless, receives cleaned DataFrames

Data Processing Layer

- cleaning_pipeline.py: Data normalization and cleaning
- validators.py: Data validation rules

Data Access Layer

- loader.py: CSV file loading with LRU caching
- registry.py: File system scanning and path resolution

Data Layer (CSV Files)

```
Data/
├── Canada/
│   ├── Montreal/listings.csv
│   ├── Toronto/listings.csv
│   └── ...
├── America/
└── Australia/
```



Component Structure

```
backend/src/
├── main.py ✓ DONE
│   └── FastAPI application initialization
├── api/v1/
│   ├── routes/
│   │   ├── analytics.py ✓ DONE
│   │   │   └── GET /analytics/?country=X&province=Y
│   │   ├── listings.py ✓ DONE
│   │   │   ├── GET /listings/countries
│   │   │   ├── GET /listings/provinces?country=X
│   │   │   └── GET /listings/?country=X&province=Y
│   └── controllers/
│       ├── analytics_controller.py ✓ DONE
│       │   └── get_analytics() - orchestrates analytics computation
│       ├── listings_controller.py ✓ DONE
│       │   └── get_listings() - orchestrates listing retrieval
```

- services/
 - └ analytics_service.py ✔ DONE
 - └ analyze_property_type_distribution()
 - └ analyze_room_type_distribution()
 - └ analyze_amenity_distribution()
 - └ analyze_price_statistics()
 - └ analyze_review_statistics()
 - └ analyze_instant_booking_stats()
- data_access/
 - └ registry.py ✔ DONE
 - └ list_countries()
 - └ list_provinces(country)
 - └ get_dataset_path(country, province, dataset)
 - └ loader.py ✔ DONE
 - └ load_csv(country, province, dataset) – with LRU cache
- data_cleaning/
 - └ cleaning_pipeline.py ✔ DONE
 - └ clean_listings() – main cleaning function
 - └ CleaningRules dataclass
 - └ DEFAULT_RULES configuration
 - └ validators.py (if exists)
- schemas/
 - └ listing_schemas.py ✔ DONE
 - └ Listing
 - └ ListingResponse
 - └ CountryListResponse
 - └ ProvinceListResponse
 - └ AnalyticsResponse
 - └ Analytics sub-models (PropertyTypeDistribution, etc.)
- config/
 - └ settings.py (placeholder for future config)
 - └ logging.py (if exists)
- utils/
 - └ cache.py (placeholder for future caching utilities)



Error Handling Flow

Exception Hierarchy

RegistryError (FileNotFoundError)

└ Raised when: country/province/dataset not found

LoaderError (RuntimeError)

↳ Raised when: CSV file cannot be read by pandas

AnalyticsControllerError (RuntimeError)

↳ Raised when: analytics cannot be computed

ListingsControllerError (RuntimeError)

↳ Raised when: listings cannot be prepared



Route Layer Error Handling

- Catches controller exceptions
- Converts to HTTPException with appropriate status codes
- Returns JSON error responses:

```
{
  "detail": "Error message"
}
```

Status Codes:

- 404: Not Found (missing data/country/province)
- 500: Internal Server Error (unexpected errors)



Caching Strategy

Caching Layers

1. Registry Cache (registry.py)
 - @lru_cache on _country_map() (maxsize=1)
 - Caches country → Path mappings
 - Cleared when DATA_ROOT changes
2. CSV Loader Cache (loader.py)
 - @lru_cache on _load_csv_from_path() (maxsize=32)
 - Caches file path → DataFrame mappings
 - refresh_cache parameter for cache invalidation
3. Future: Response Cache (utils/cache.py)
 - Can add memoization for API responses
 - TTL-based expiration



Data Flow Diagram

CSV File (Data/Canada/Montreal/listings.csv)



```
registry.get_dataset_path()
→ Resolves absolute path to CSV
```



```
loader.load_csv()
→ Reads CSV with pandas.read_csv()
→ Returns raw pandas DataFrame
→ Cached for subsequent requests
```



```
cleaning_pipeline.clean_listings()
→ Normalizes columns, converts types, handles nulls
→ Returns cleaned DataFrame
```



For Analytics

analytics_
controller

1. Compute analytics
2. Build Analytics Response

For Listings

listings_
controller

1. Convert to Pydantic models
2. Return ListingResponse



JSON Response
(validated by
Pydantic schemas)



Implementation Checklist



Already Done

- ☒ FastAPI application setup with CORS
- ☒ Router structure (analytics, listings)
- ☒ Controller layer (analytics, listings)
- ☒ Data access layer (registry, loader with caching)
- ☒ Data cleaning pipeline
- ☒ Analytics service functions
- ☒ Pydantic schemas for all responses
- ☒ Error handling with custom exceptions
- ☒ LRU caching for CSV loading
- ☒ Country/province listing endpoints



Future Enhancements

- ☐ Response caching with TTL
- ☐ Configuration management (settings.py)
- ☐ Logging configuration
- ☐ Database integration (optional)
- ☐ API authentication/authorization
- ☐ Rate limiting
- ☐ Pagination for large listing responses
- ☐ Async CSV loading for better performance
- ☐ Data validation enhancements
- ☐ Unit tests for all layers
- ☐ Integration tests
- ☐ API documentation enhancements
- ☐ OpenAPI schema improvements



Design Patterns

1. Layered Architecture

- **API Layer:** HTTP concerns (routes, schemas)
- **Controller Layer:** Business logic orchestration

- **Service Layer:** Pure computation functions
- **Data Access Layer:** File system operations
- **Data Layer:** CSV files

2. Dependency Injection

- Controllers depend on services and data access
- Services are stateless and pure functions
- Easy to test and mock

3. Caching Strategy

- LRU cache for expensive operations (CSV reading)
- Registry caching for directory scans
- Cache invalidation via refresh_cache parameter

4. Error Handling

- Custom exception hierarchy
- Graceful error propagation
- HTTP error codes mapped to exceptions

5. Data Cleaning Pipeline

- Configurable cleaning rules per dataset
- Pluggable cleaning steps
- Extensible for new datasets



Configuration

Environment Variables (Future)

```
DATA_ROOT=/path/to/Data  
LOG_LEVEL=info
```

```
# Override default Data/ path  
# Logging level (debug/info/warning/error)
```

CACHE_TTL_SECONDS=3600

API_HOST=0.0.0.0

API_PORT=8000

Cache expiration time

Server host

Server port



API Endpoints Summary

Method	Endpoint	Description	Parameters
GET	<code>/api/v1/listings/countries</code>	List available countries	None
GET	<code>/api/v1/listings/provinces</code>	List provinces for a country	<code>country</code> (query)
GET	<code>/api/v1/listings/</code>	Get listings for country/province	<code>country</code> , <code>province</code> , <code>dataset</code> (optional)
GET	<code>/api/v1/analytics/</code>	Get analytics for country/province	<code>country</code> , <code>province</code> (optional), <code>dataset</code> (optional)
GET	<code>/</code>	API information	None
GET	<code>/health</code>	Health check	None



Deployment Considerations

- **Development:** `uvicorn` with `reload=True` for hot reloading
- **Production:** Use ASGI server like Gunicorn with Uvicorn workers
- **Environment:** Set `DATA_ROOT` environment variable for production
- **Caching:** Consider Redis for distributed caching in multi-instance deployments
- **Monitoring:** Add health check endpoints and logging
- **Security:** Add authentication, rate limiting, input validation